

Design Document for the Unified Supply Management System (OMS, TMS, DMS)

Table of Contents:

- 1.Introduction
- 2.Objective
- 3.System Overview
- 4.Technology Stack
- 5.Detailed Architecture
 - OMS (Order Management System)
 - TMS (Transport Management System)
 - DMS (Delivery Management System)
- 6.Data Flow and Kafka Integration
- 7.Database Design and ER Diagrams
- 8.API Design
- 9.Security
- 10.References

1.Introduction

This document describes the design of the Unified Supply Chain Management System that integrates the Order Management System (OMS), Transportation Management System (TMS), and Distribution Management System (DMS). The project aims to streamline the entire supply chain process—from order placement to final delivery—through seamless integration of these modules.

2.Objective

The objective is to build a scalable, modular system that enhances operational efficiency and data flow between the OMS, TMS, and DMS using microservice architecture, Spring Boot, Kafka, and a relational database. Each module will handle specific business logic while communicating via asynchronous messaging with Kafka.

3.System Overview

Order Management System (OMS)

- Main Responsibilities: Handle customer orders, validate them, check inventory, and forward the request to TMS for shipment.
- Input: Customer orders via API.
- Output: Kafka messages to the TMS for shipment creation.

Transportation Management System (TMS)

- Main Responsibilities: Handle shipment creation, route optimization, and carrier assignment based on orders received from the OMS.
- Input: Kafka messages from the OMS.
- Output: Kafka messages to the DMS for warehouse and shipment updates.

Distribution Management System (DMS)

- Main Responsibilities: Manage warehouse inventory, track shipments, and handle product returns.
- Input: Kafka messages from TMS.
- Output: Inventory status, shipment details, and product returns.

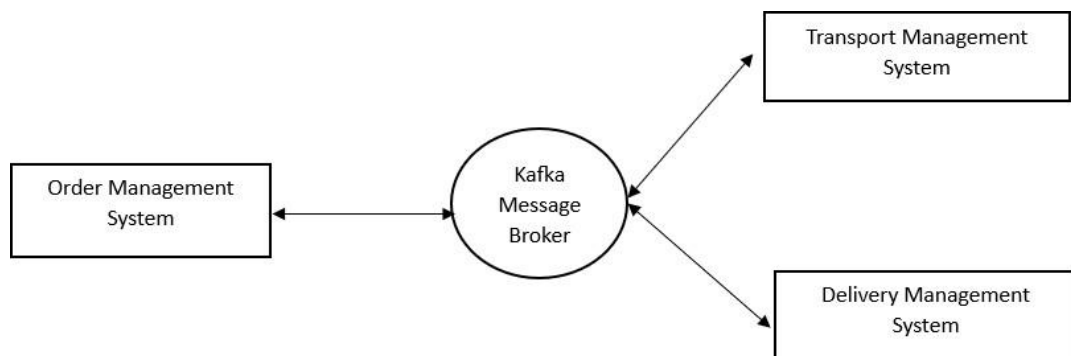
4. Technology Stack

- **Java 17:** Core programming language for business logic.
- **Spring Boot:** Framework for building REST APIs and service layers.
- **Apache Kafka:** Message broker for asynchronous communication between OMS, TMS, DMS.
- **MySQL:** Relational database for persistent storage.
- **JPA:** ORM for data management.
- **Docker:** For containerizing services.
- **Swagger:** API documentation.
- **Spring Security:** Role-based access control for APIs.

5. Detailed Architecture

System Architecture of OMS, TMS, DMS:

Order Management System (OMS), Transportation Management System (TMS), and Distribution Management System (DMS), here's a detailed system architecture that shows the interactions, communication, and overall structure of the unified system. The architecture leverages **Kafka** for communication between the different services.



1.OMS (Order Management System)

Responsibilities:

- Capture and validate customer orders.
- Handle customer and order data.
- Interact with **TMS** for transportation and shipment creation.

Components:

- **REST API (Controller):**
 - Handles incoming HTTP requests for placing orders, viewing order status, and retrieving order information.
- **Service Layer:**
 - Business logic for handling order placement, including saving orders to the database.
 - Sends order events to Kafka topic for TMS and DMS to process.
- **Repository Layer:**
 - Manages data persistence using JPA for the Orders, Customers, and Order Items entities.
- **Kafka Producer:**
 - Sends messages to Kafka topic order-events when a new order is placed.

2. TMS (Transportation Management System)

Responsibilities:

- Route planning and optimization.
- Carrier management and assignment.
- Freight auditing and shipment tracking.

Components:

- **REST API (Controller):**
 - Exposes endpoints for managing shipments, assigning carriers, and retrieving real-time tracking.
- **Service Layer:**
 - Listens to Kafka events from the OMS (order-events) for new orders.
 - Processes route optimization based on distance and delivery location.
 - Manages carrier assignment and shipment creation.
 - Sends shipment details to Kafka topic shipment-events for DMS to process.
- **Repository Layer:**
 - Manages data persistence for Shipments, Routes, and Carriers entities.
- **Kafka Consumer:**
 - Listens to Kafka topic order-events for new orders.
- **Kafka Producer:**
 - Sends shipment-related events to Kafka topic shipment-events.

3. DMS (Distribution Management System)

Responsibilities:

- Warehouse management (inventory control, stock levels).
- Handle product returns and restocking.
- Real-time tracking of stock and shipments across warehouses.

Components:

- **REST API (Controller):**
 - Provides endpoints for managing warehouse data, inventory updates, and product returns.
- **Service Layer:**
 - Listens to Kafka events from TMS (shipment-events) for new shipments.
 - Updates inventory levels in the warehouse.
 - Handles product returns and restocking.
 - Uses warehouse automation to optimize inventory levels and manage stock.
- **Repository Layer:**
 - Manages data persistence for Warehouses, Inventory, and Product Return.
- **Kafka Consumer:**
 - Listens to Kafka topic shipment-events for new shipments to update the warehouse system.

Overall System Flow

1. Order Creation (OMS):

- A customer places an order via the **OMS API**.
- OMS validates and saves the order, then produces a Kafka message to the order-events topic.

2. Order Processing (TMS):

- **TMS Kafka Consumer** listens to order-events and retrieves the new order.
- TMS optimizes the route, assigns a carrier, and creates a shipment.
- TMS then produces a Kafka message to the shipment-events topic containing shipment details.

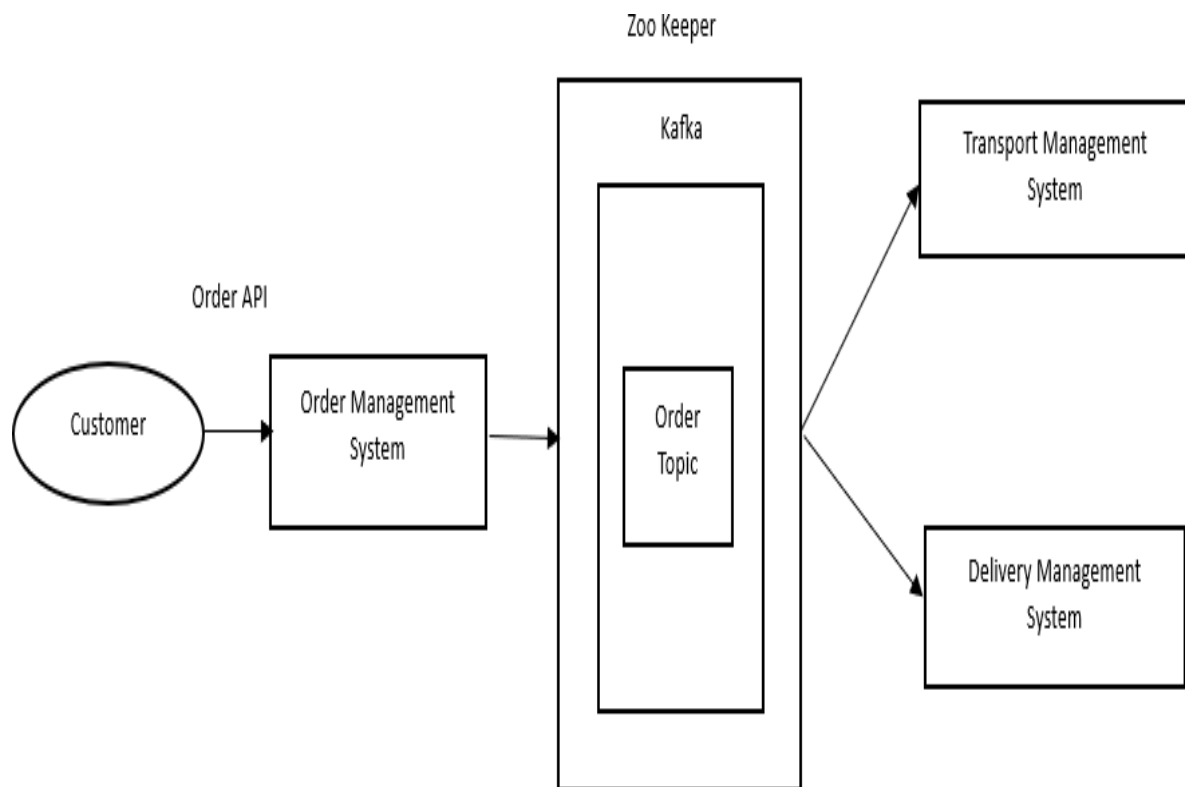
3. Shipment and Inventory Updates (DMS):

- **DMS Kafka Consumer** listens to shipment-events and processes the shipment for inventory control.
- DMS updates stock levels, tracks the shipment, and handles product returns if necessary.

Architecture Diagram Breakdown

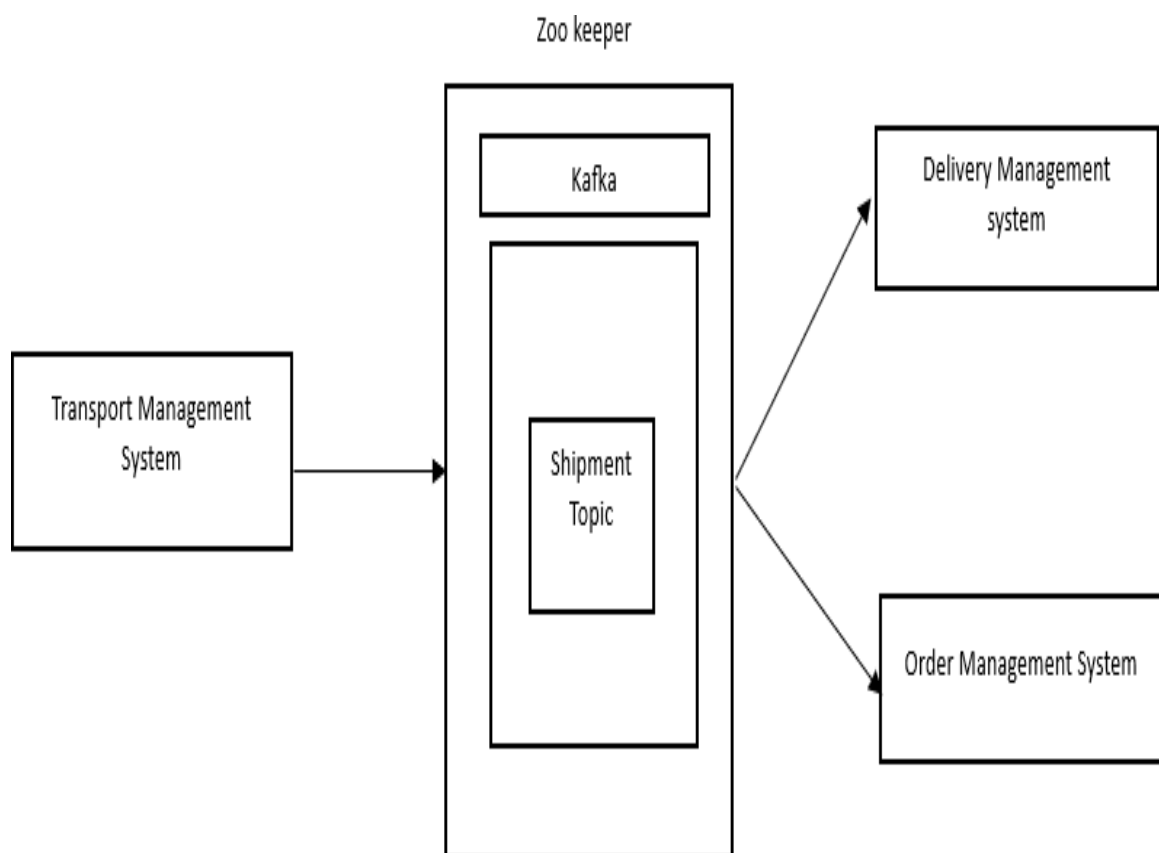
High-Level Architecture

- **Frontend Clients (e.g., Web or Mobile):** Interact with OMS APIs to place orders.
- **Order Management System (OMS):**
 - REST API (Order Creation, Order Status)
 - Kafka Producer to send orders to order-events.
 - MySQL Database for orders and customer data.



- **Transportation Management System (TMS):**

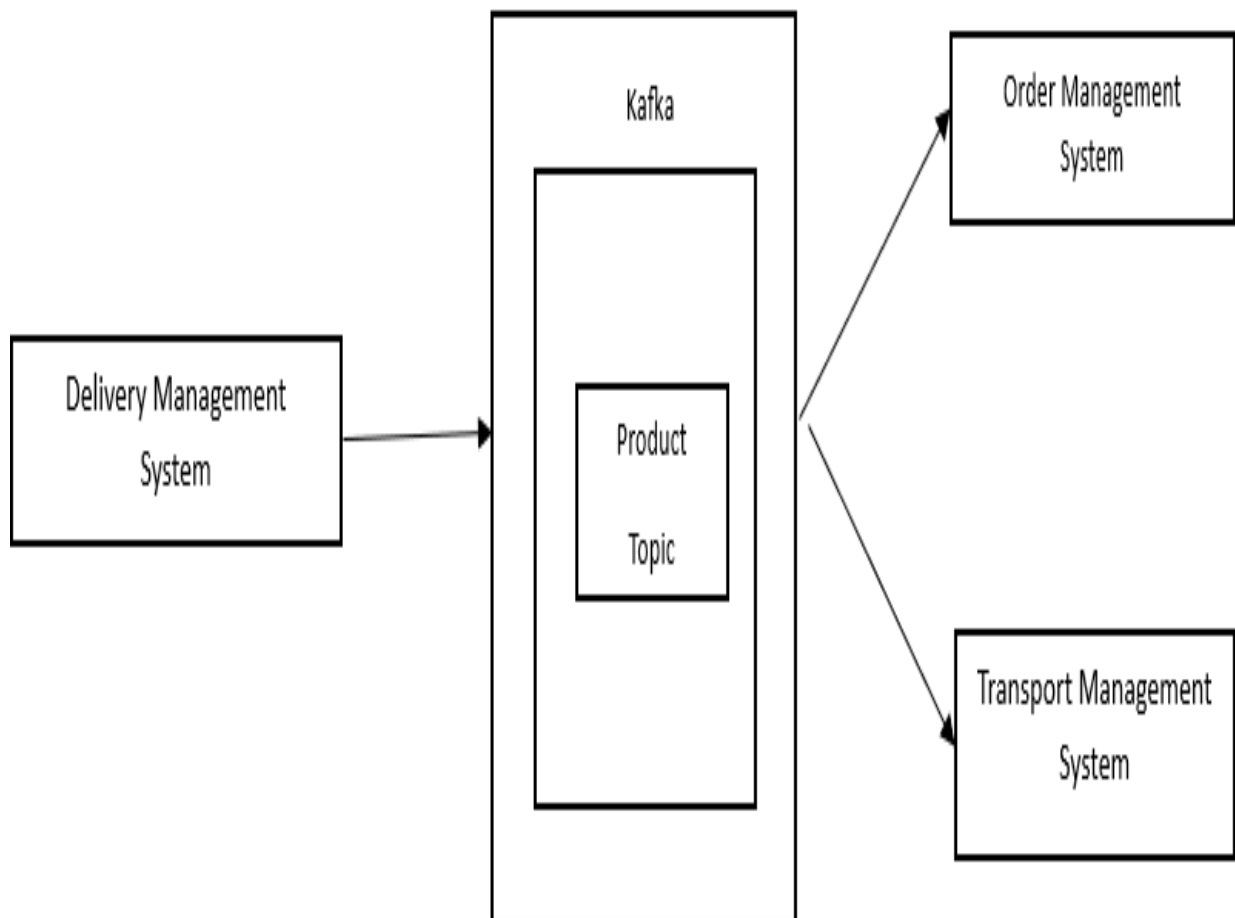
- REST API (Shipment Management, Carrier Assignment)
- Kafka Consumer for order-events.
- Route Optimization Engine.
- Kafka Producer to send shipments to shipment-events.
- MySQL Database for shipments, carriers, routes.



- **Distribution Management System (DMS):**

- REST API (Warehouse Management, Inventory, Product Returns)
- Kafka Consumer for shipment-events.
- Kafka Producer to send shipments to shipment-events.
- MySQL Database for inventory, warehouses, returns.

Zoo Keeper



Detailed Module Interaction

1. Order Management System (OMS):

- Responsible for handling customer orders.
- Communicates with TMS via Kafka to create shipments.

2. Transportation Management System (TMS):

- Listens for new orders from OMS, optimizes routes, assigns carriers.
- Updates DMS about shipment statuses via Kafka.

3. Distribution Management System (DMS):

- Listens for shipment events from TMS and updates inventory accordingly.
- Manages returns and ensures accurate stock levels.

6.Data Flow and Kafka Integration

Order Flow (OMS -> TMS -> DMS)

1. Order Created (OMS):

- A customer places an order through OMS, which sends the order data to Kafka (order-events).

2. Shipment Creation (TMS):

- TMS consumes the order-events message and processes the shipment creation, route optimization, and carrier assignment. It then sends a message to Kafka (shipment-events).

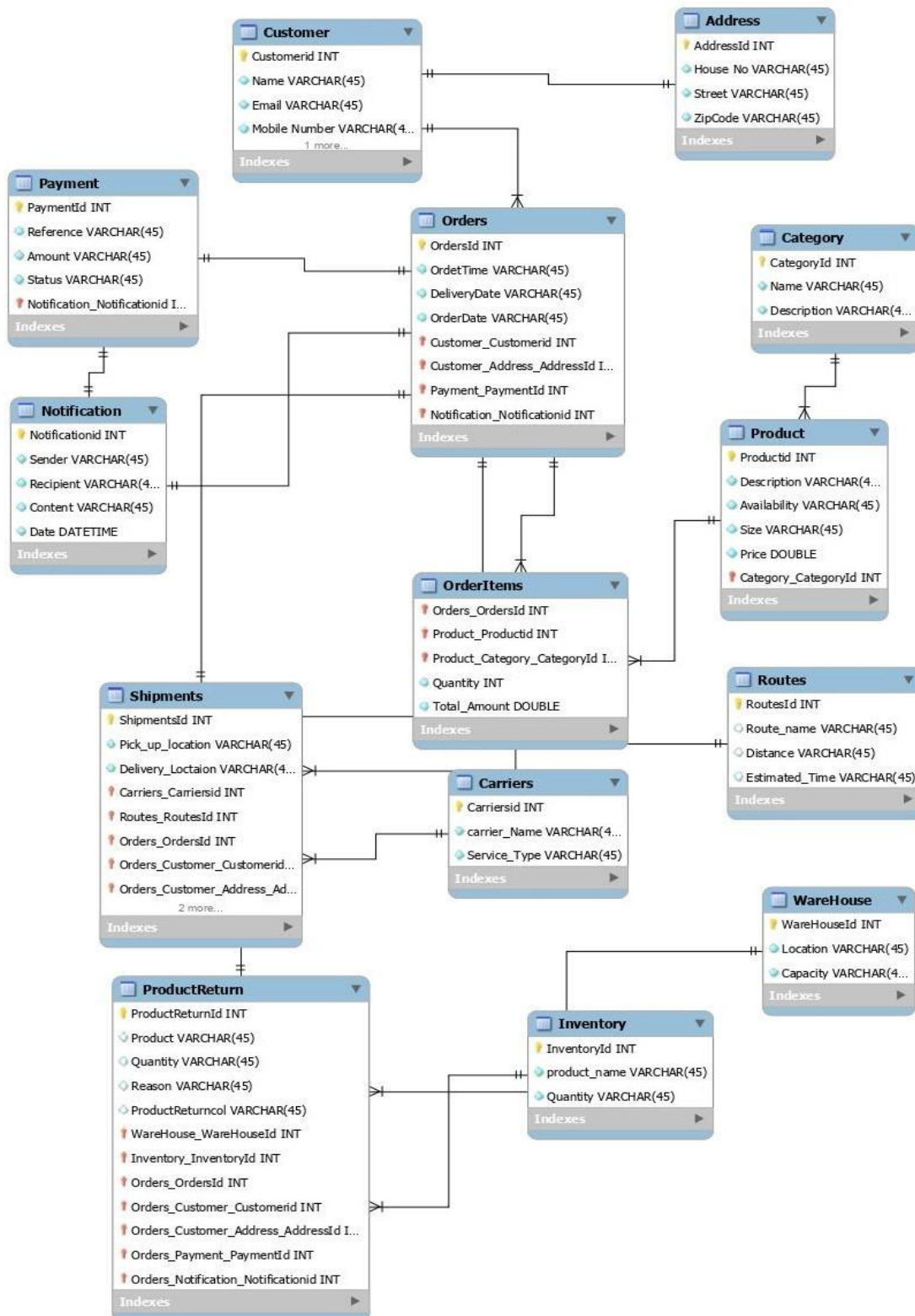
3. Inventory Update (DMS):

- DMS listens to shipment-events and updates the warehouse stock, tracks shipment progress, and handles returns.

Kafka Topic Definitions:

- **order-events:** Carries order details from OMS to TMS.
- **shipment-events:** Carries shipment details from TMS to DMS.

7.Database Design and ER Diagram:



Entities for OMS:

- Orders: Order ID, customer info, status, etc.
- OrderItems: Order line items related to Orders.

Entities for TMS:

- Shipments: Shipment ID, pickup and delivery locations, carrier details.
- Carriers: Carrier ID, name, service type, related shipments.
- Routes: Route details with related shipments.

Entities for DMS:

- Warehouses: Warehouse details, inventory.
- Inventory: Tracks product stock levels.
- ProductReturns: Manages product returns.

8.API Design

OMS API Endpoints

- POST /orders: Place a new order.
- GET /orders/{id}: Retrieve order details.

TMS API Endpoints

- POST /shipments: Create a new shipment.
- GET /shipments/{id}: Retrieve shipment details.

DMS API Endpoints

- POST /returns: Handle a product return.
- GET /inventory/{productId}: Check inventory levels.

9. Security

- **Spring Security:** Role-based access control for different users (Admin, Customer, Warehouse Manager).
- **JWT Tokens:** Used for authentication across services.
- **OAuth2:** For third-party integrations like payment gateways.

10. References

1. **Spring Boot Documentation:** <https://spring.io/projects/spring-boot>
2. **Kafka Documentation:** <https://kafka.apache.org/documentation/>
3. **JPA/Hibernate:** <https://hibernate.org/>
4. **PostgreSQL Documentation:** <https://www.postgresql.org/docs/>
5. **Docker:** <https://www.docker.com/>

Conclusion

The design document outlines the architecture, business logic, and communication mechanisms between OMS, TMS, and DMS using Kafka. Each module is independently deployable and can scale as needed.